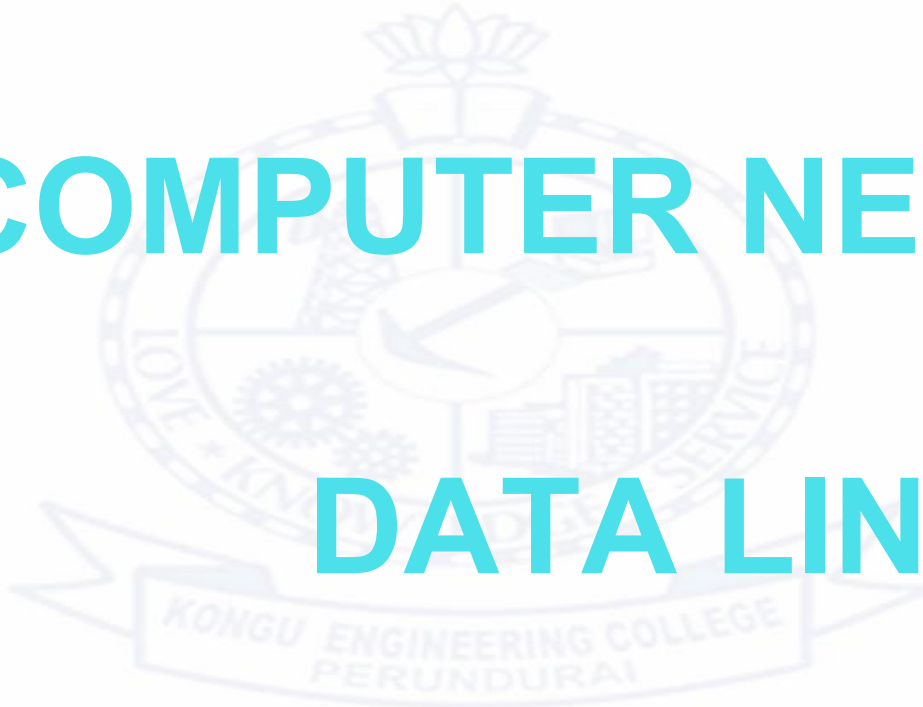


The logo of Kongu Engineering College is a circular emblem. It features a central gear with a lotus flower on top. The text 'KONGU ENGINEERING COLLEGE' is written around the top half of the circle, and 'PERUNDURAI' is at the bottom. A banner across the middle contains the motto 'LOVE * KNOWLEDGE * SERVICE'.

COMPUTER NETWORKS

UNIT-2

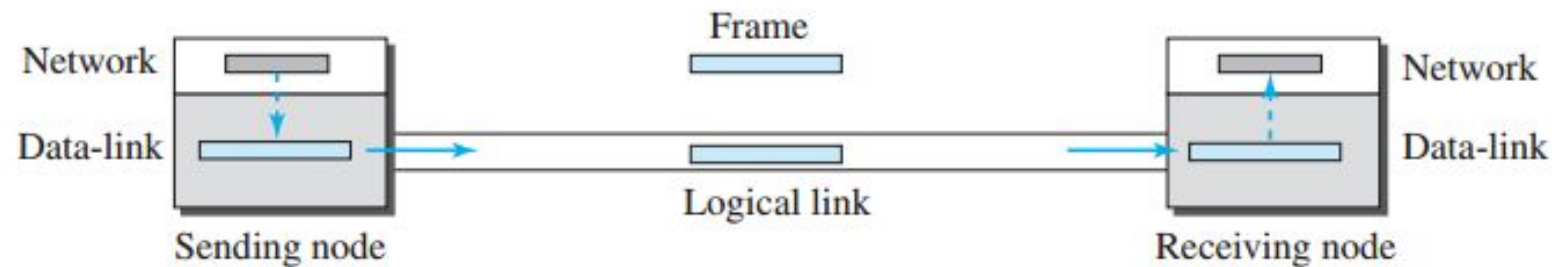
DATA LINK LAYER

Estd : 1984

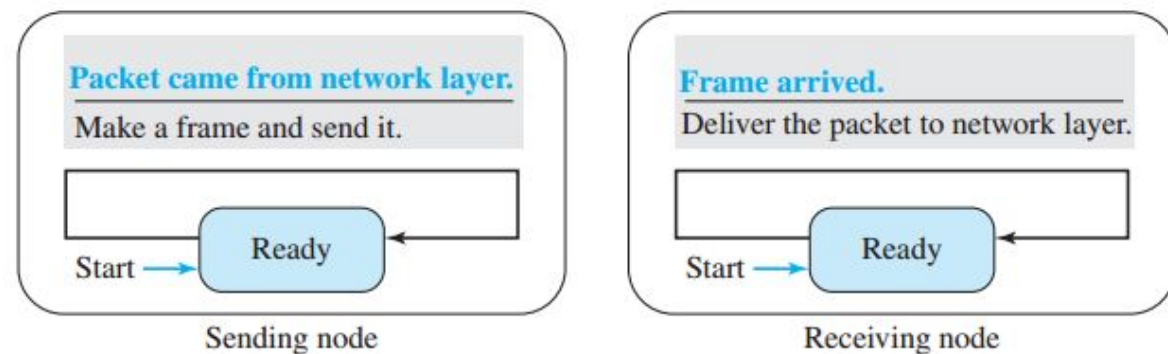
Data link layer protocols

1. Simple protocol:

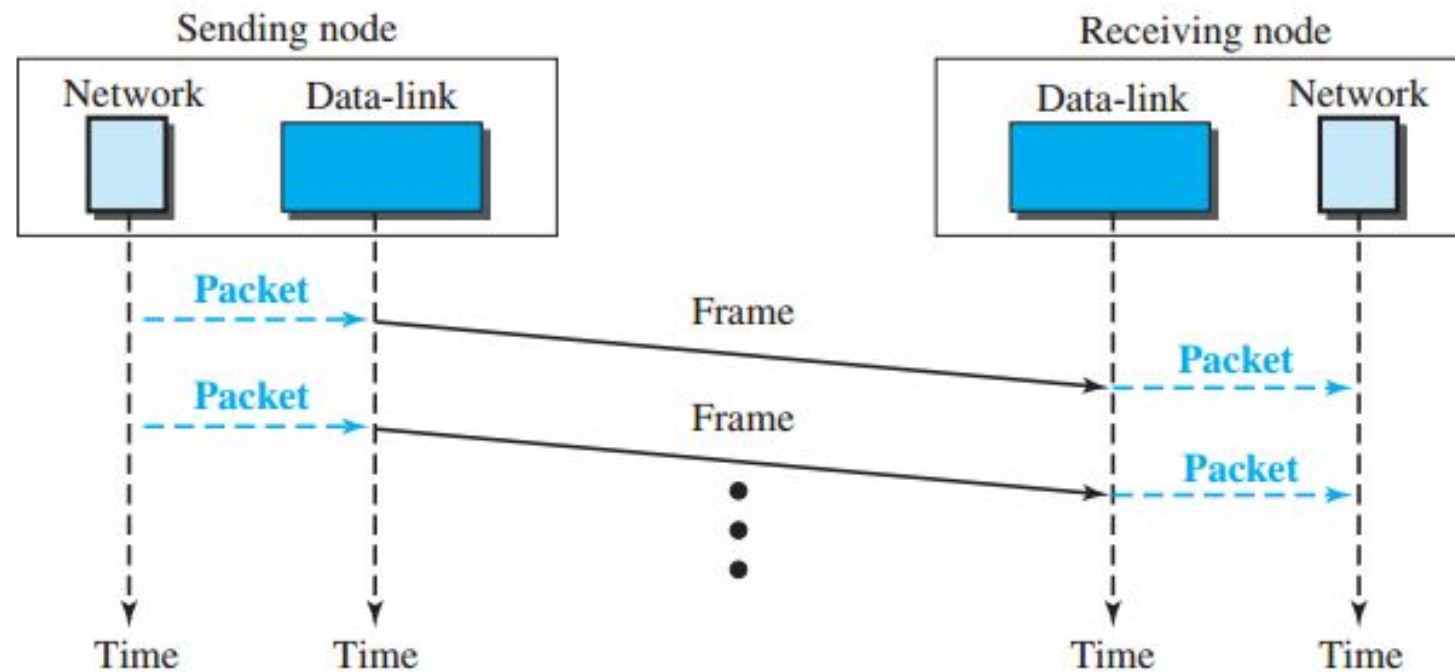
- neither flow nor error control.
- assume that the receiver can immediately handle any frame it receives.



- FSMs for the simple protocol:

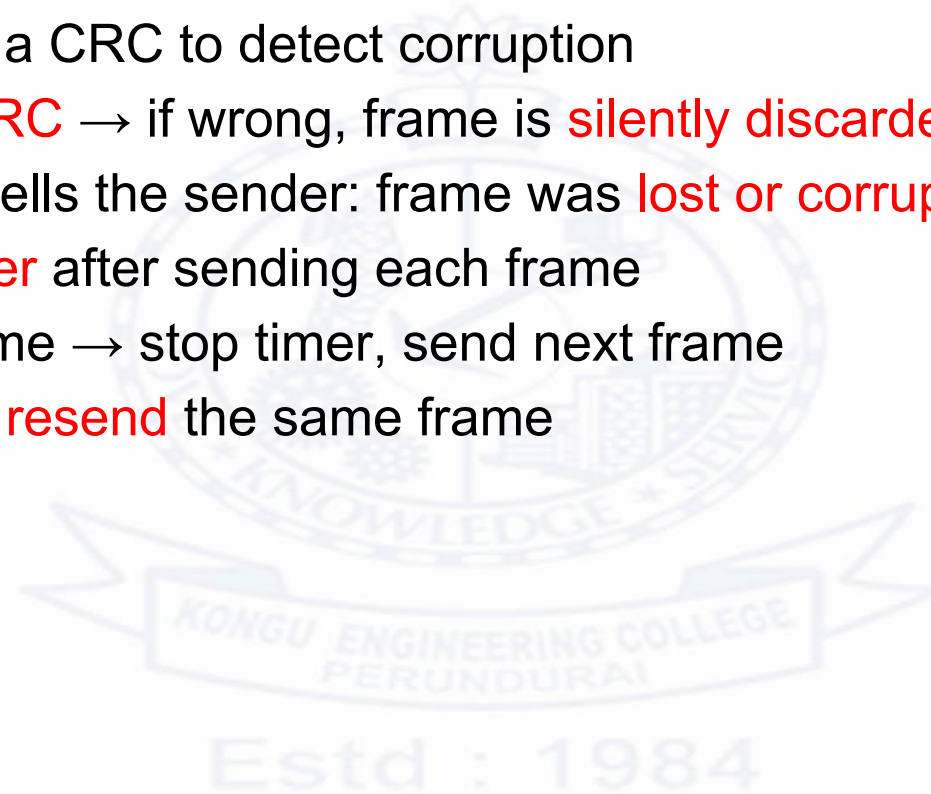


- The data-link layer at the sender gets a packet from its network layer, makes a frame out of it, and sends the frame.
- The data-link layer at the receiver receives a frame from the link, extracts the packet from the frame, and delivers the packet to its network layer.

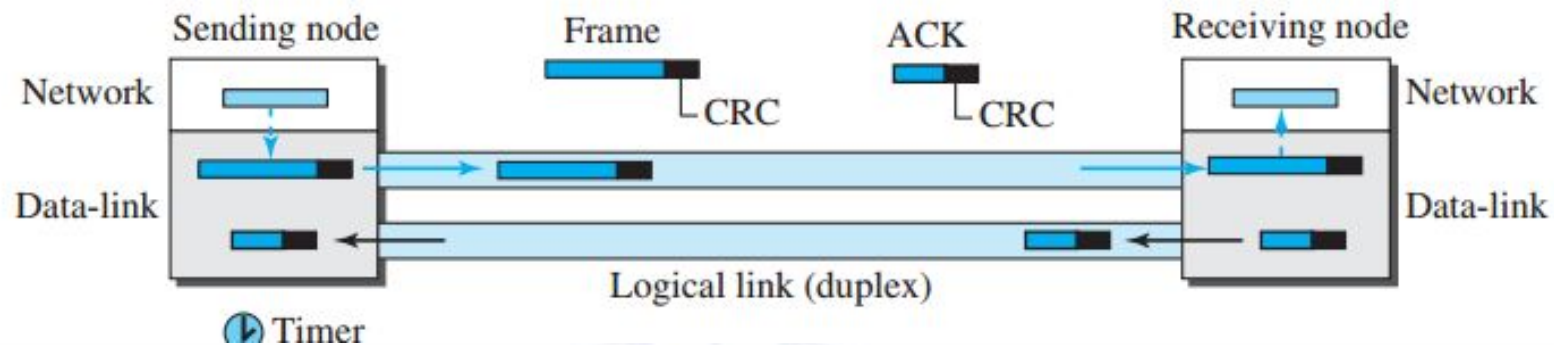


2. Stop and wait protocol:

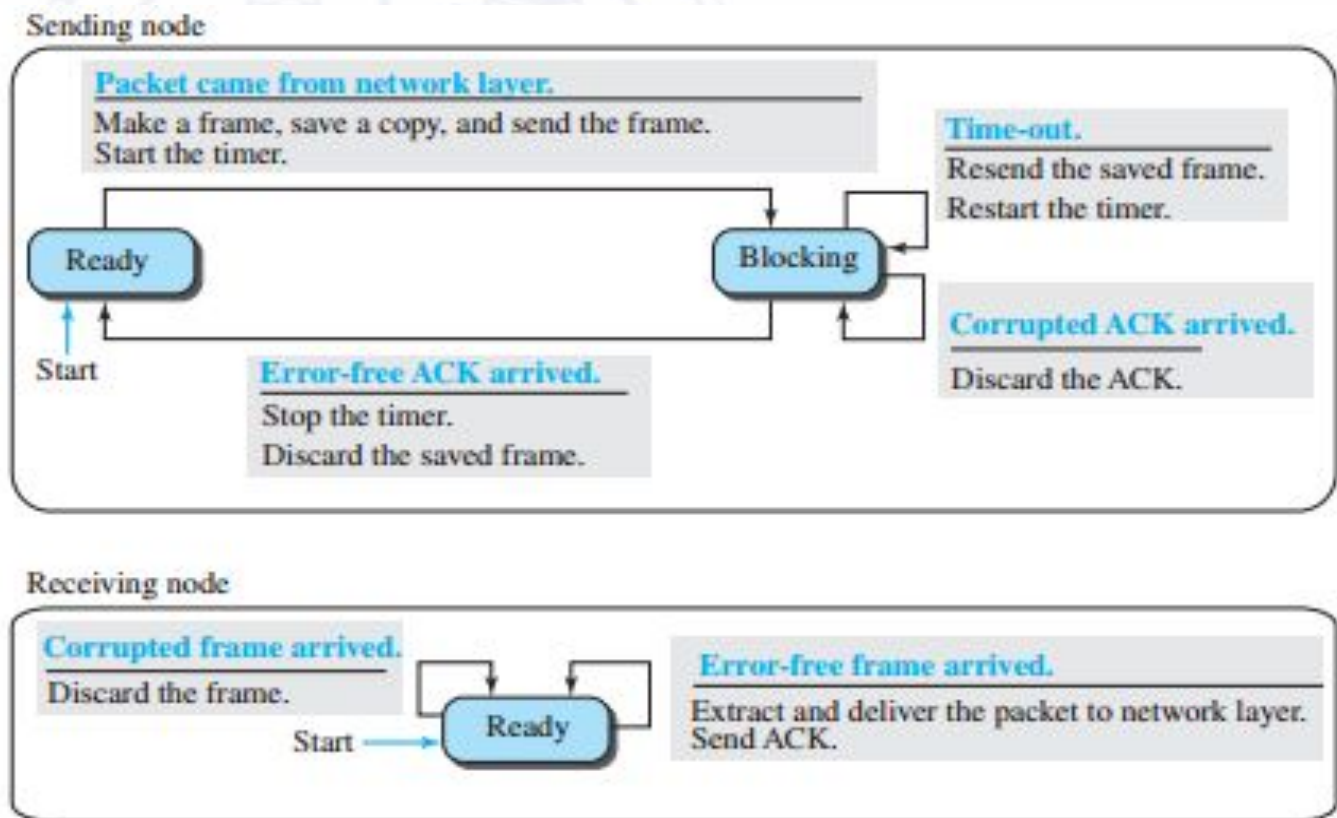
- Combines flow control + error control
- Sender sends **one frame**, then waits for ACK before sending the next
- Each frame carries a CRC to detect corruption
- Receiver checks **CRC** → if wrong, frame is **silently discarded**
- Receiver's silence tells the sender: frame was **lost or corrupted**
- Sender starts a **timer** after sending each frame
 - ACK arrives in time → stop timer, send next frame
 - Timer expires → **resend** the same frame

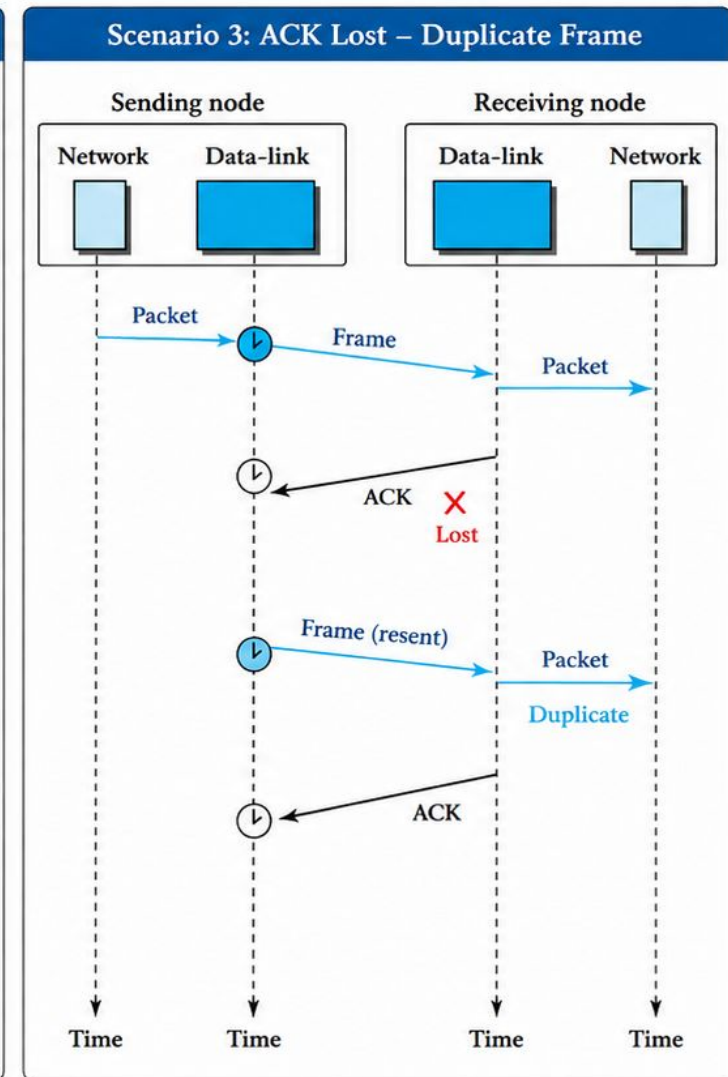
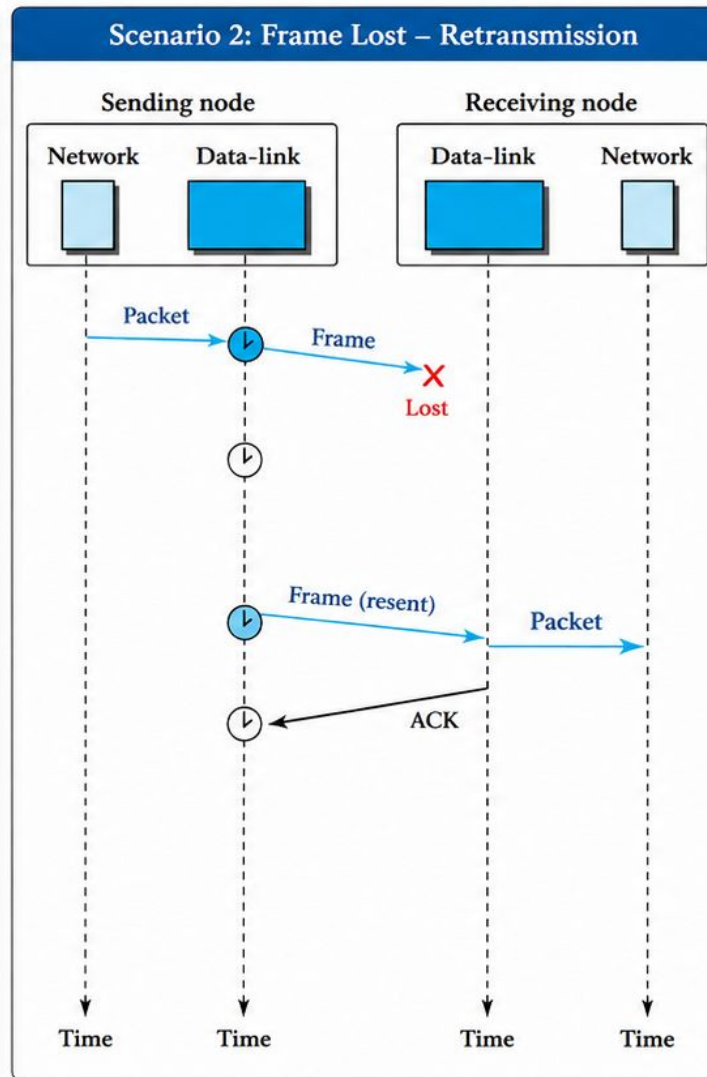
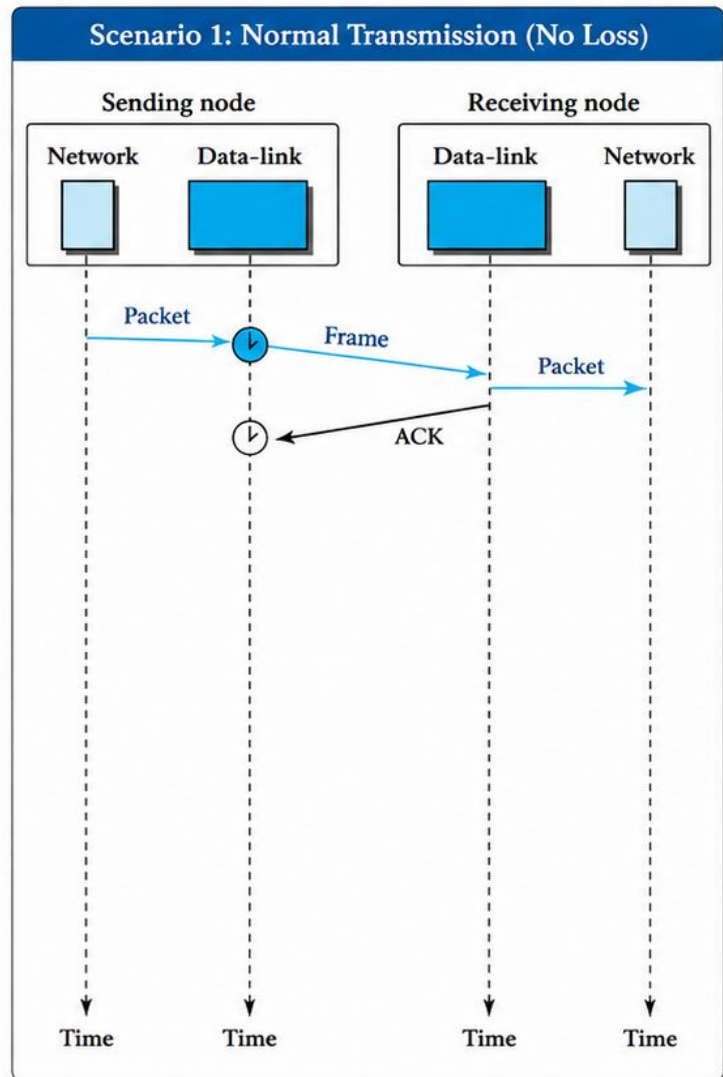


- **Sender:**
- **Ready State**
 - Waiting for a packet from the network layer
 - On packet arrival: create frame → save copy → start timer → send frame → move to **Blocking**
- **Blocking State** — three possible events:
 - **Time-out** → resend saved frame, restart timer
 - **Corrupted ACK arrives** → discard it (stay in Blocking) Wait → Timeout → Resend
 - **Error-free ACK arrives** → stop timer, discard saved copy → move to **Ready**
- **Receiver:**
- Always in the **Ready** state. Two possible events:
 - **Error-free frame arrives** → deliver message to network layer, send ACK
 - **Corrupted frame arrives** → discard the frame (no ACK)



- FSM for stop and wait protocol:

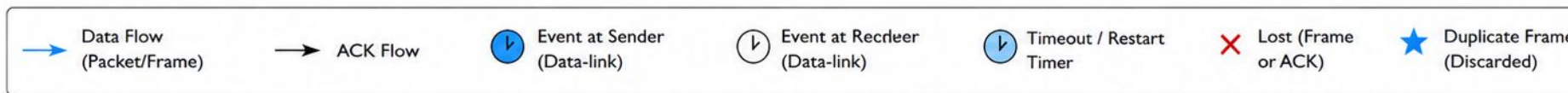
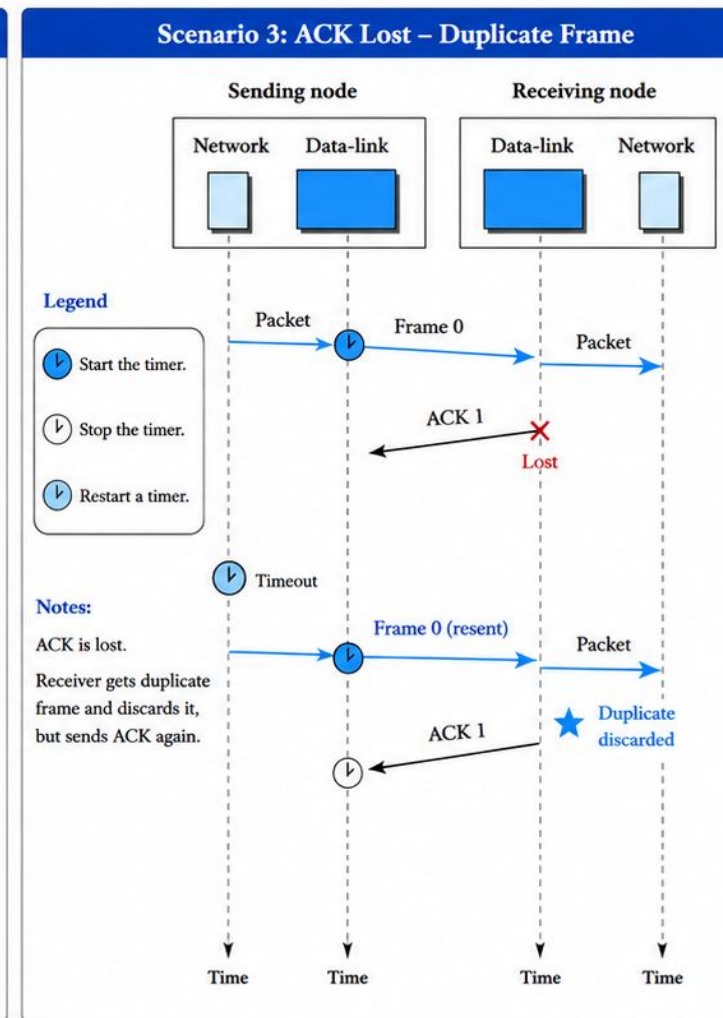
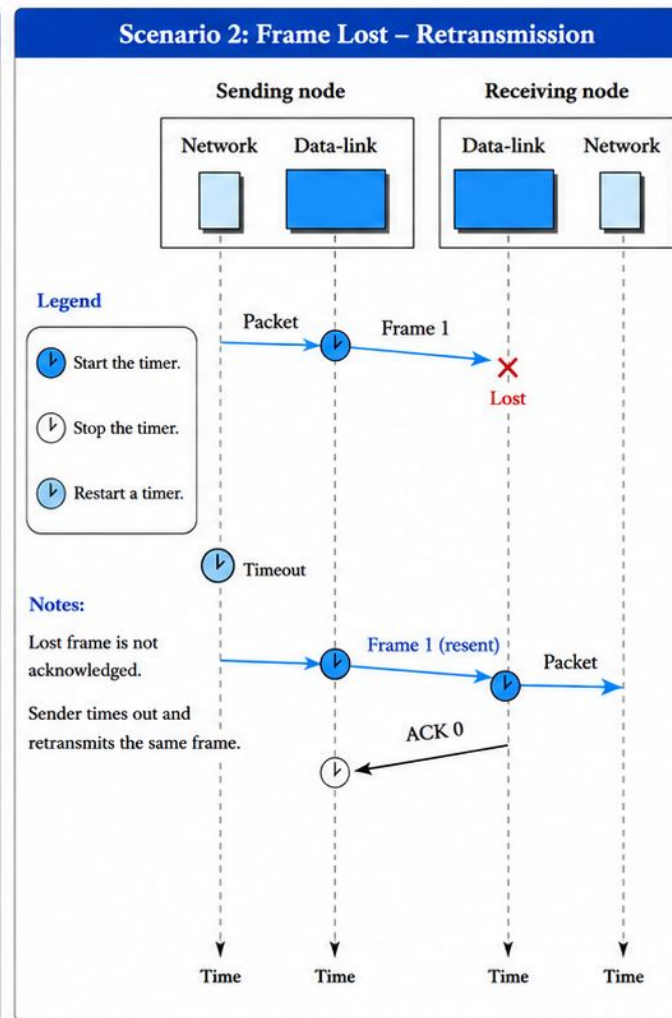
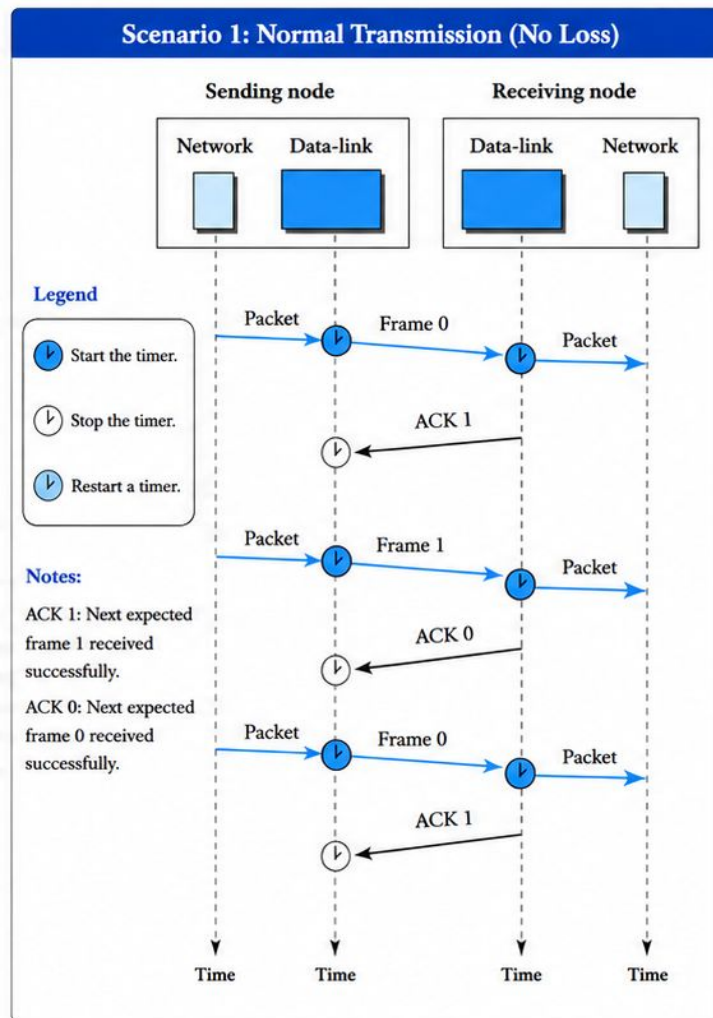




Data Flow (Packet/Frame)
 ACK Flow
 Event at Sender (Data-link)
 Event at Receiver (Data-link)

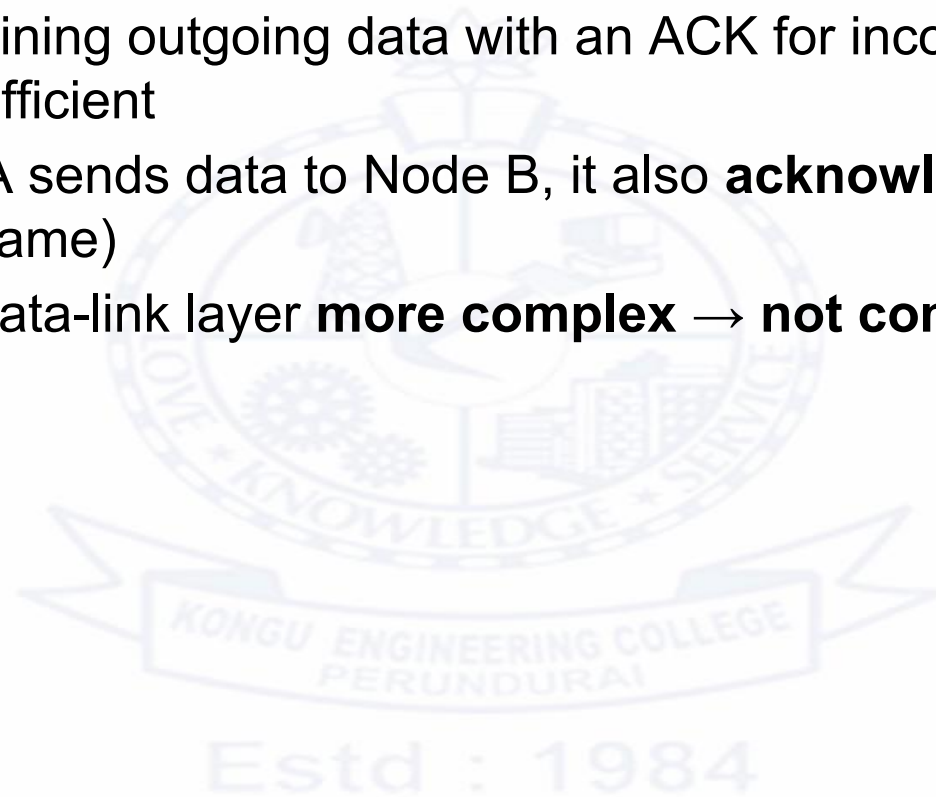
Sequence and Acknowledgment Numbers

- **Problem:** duplicate packets (from corrupted/resent frames) must be avoided — e.g., ordering an item twice by mistake
- **Fix:** add **sequence numbers** to data frames and **acknowledgment numbers** to ACK frames
- Numbering scheme is simple — just alternates:
 - **Sequence numbers:** 0, 1, 0, 1, 0, 1, ...
 - **Acknowledgment numbers:** 1, 0, 1, 0, 1, 0, ...
- **Key rule:** an **ACK number** always indicates the **sequence number of the *next* frame expected**
- ACK numbering **starts at 1** (since it points to the next frame, not the one just received)

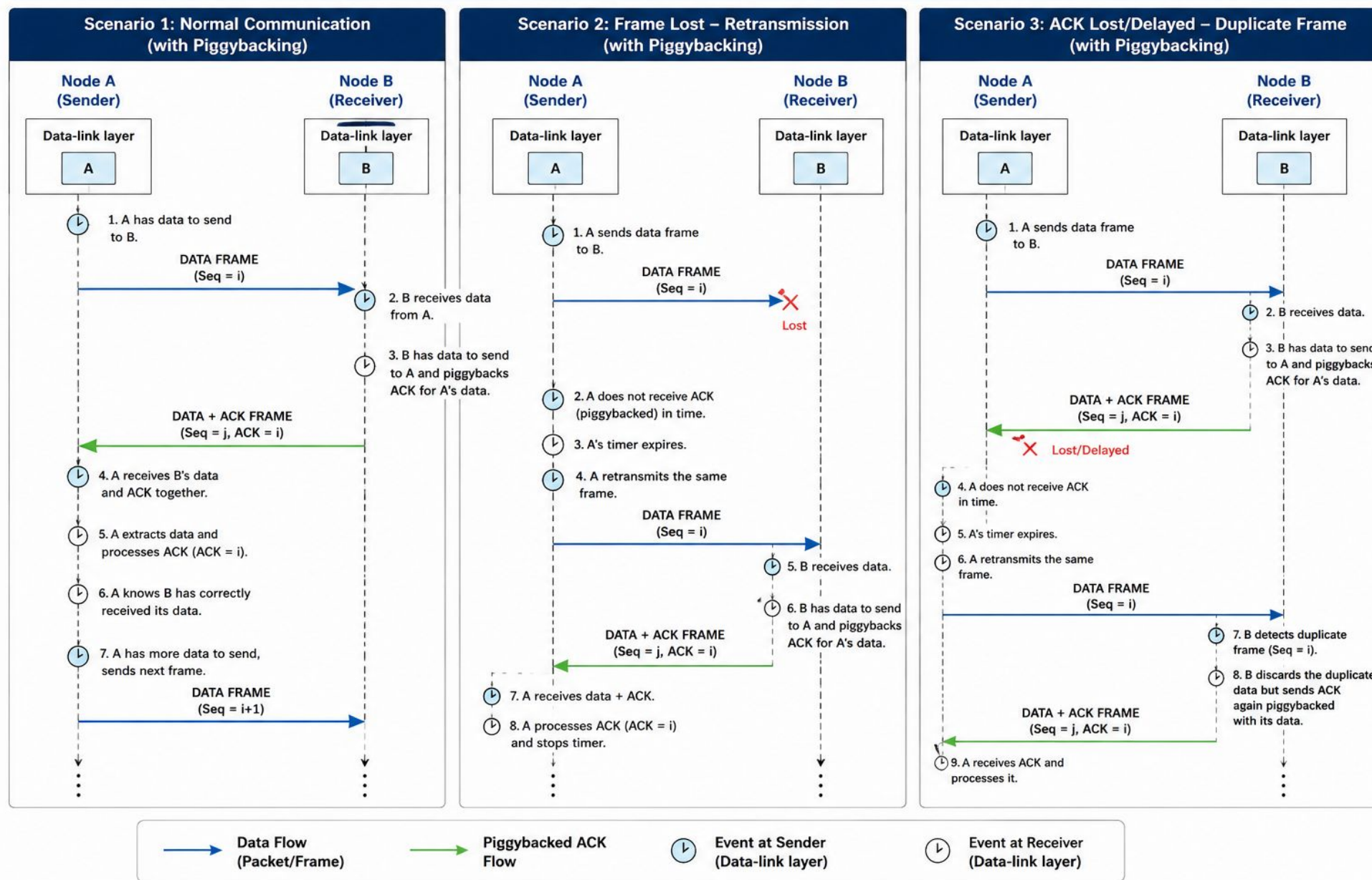


Piggybacking

- Protocols discussed so far are **unidirectional** — data flows one way, ACKs travel the other way
- **Piggybacking** = combining outgoing data with an ACK for incoming data, to make communication more efficient
- Example: while Node A sends data to Node B, it also **acknowledges** data received from Node B (in the same frame)
- Trade-off: makes the data-link layer **more complex** → **not commonly used** in practice



PIGGYBACKING: COMBINING DATA WITH ACK IN THE SAME FRAME



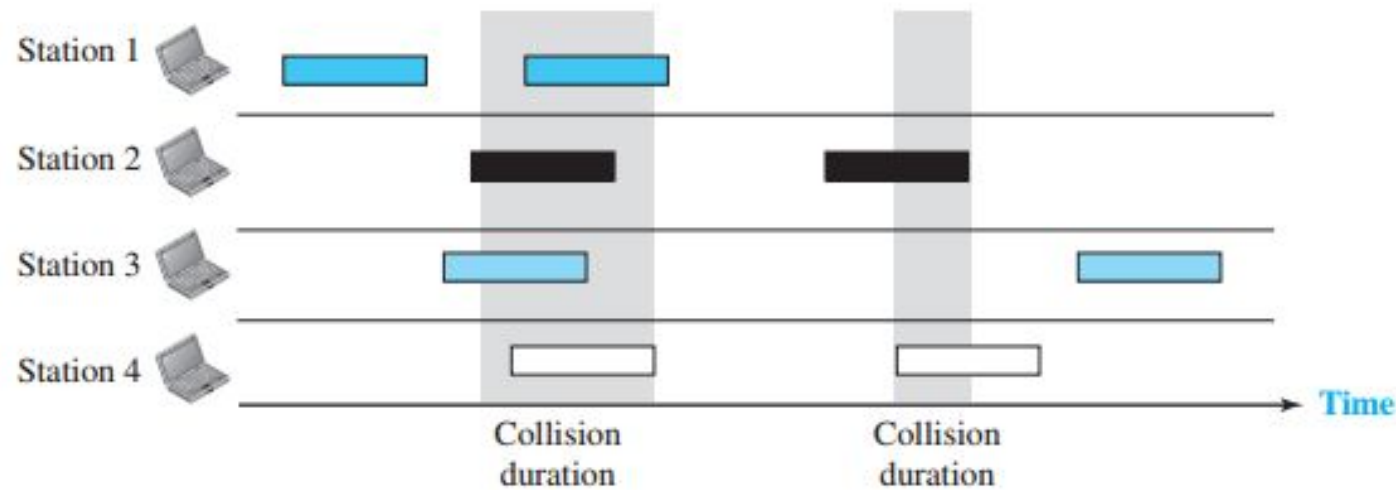
Random access (contention) methods

- No station has priority — none controls another
- Each station decides for itself **whether to send**, based on whether the medium is **idle or busy**
- Two features give the method its name:
 - **Random access** — no scheduled time slot; transmission timing is random
 - **Contention** — no rule decides who sends next; stations **compete** for the medium
- Any station can access the medium freely — but if **more than one station sends at once**, a **collision** occurs
- A collision **destroys or corrupts** the frames involved

- Evolution of Random Access Methods:
 - Originated from **ALOHA**, using a simple procedure called **Multiple Access (MA)**
 - Improved with **CSMA** (Carrier Sense Multiple Access) — station senses the medium before transmitting
 - CSMA evolved into two parallel methods:
 - **CSMA/CD** (Collision Detection) — defines what a station does *after* a collision is detected
 - **CSMA/CA** (Collision Avoidance) — tries to *prevent* a collision from happening in the first place
- **Evolution path:** ALOHA → MA → CSMA → CSMA/CD & CSMA/CA

Pure ALOHA

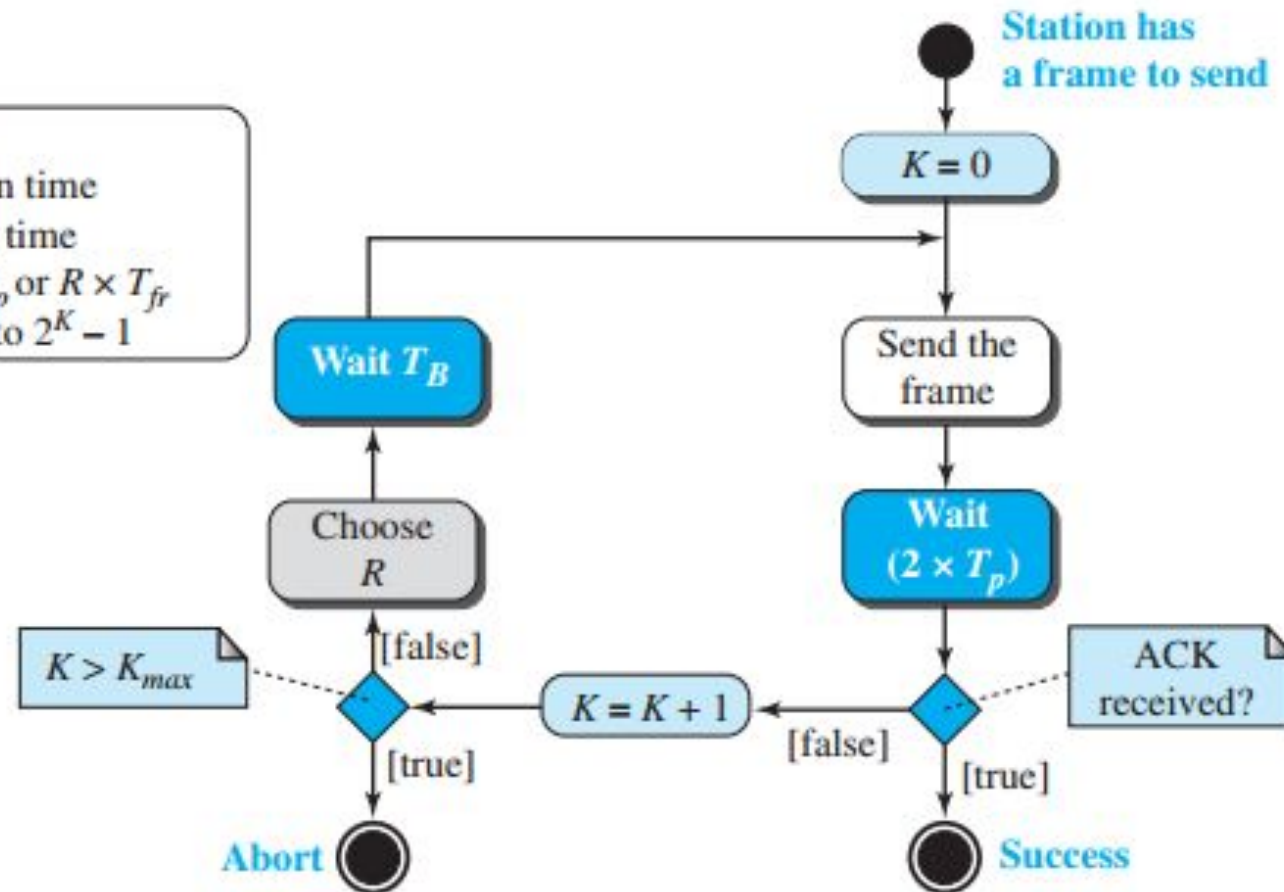
- Each station sends a frame **whenever it has one ready** (true multiple access, no coordination)
- Since all stations share **one single channel**, **collisions** can occur between frames sent by different stations



- Retransmission Strategy:
 - Relies on **acknowledgments (ACKs)** from the receiver for every frame sent
 - If no ACK arrives within a **time-out period**, the station assumes the frame (or ACK) was destroyed → **resends** the frame
 - If two+ stations collide and all resend immediately, they'll **collide again**
 - Fix: each station waits a **random back-off time (T_B)** before resending — randomness reduces repeat collisions
 - To avoid endless retransmission: give up after **K_{max}** attempts (usually **15**)
- **Time-out period** = $2 \times T_p$ (twice the maximum propagation delay between the two farthest stations)
- **T_B** (back-off time) is random and grows with each failed attempt
- Common method: **binary exponential backoff**
 - Choose random multiplier **R = 0 to $2^K - 1$**
 - $T_B = R \times T_p$ (or T_{fr} Transmission time)
 - Random range **doubles after each collision**, spreading retries out over time.

Legend

K : Number of attempts
 T_p : Maximum propagation time
 T_{fr} : Average transmission time
 T_B : (Backoff time): $R \times T_p$ or $R \times T_{fr}$
 R : (Random number): 0 to $2^K - 1$



- **Vulnerable time** = the window during which a collision can occur for a frame being sent
- Each frame takes T_{fr} seconds to transmit (fixed-length frames assumed)
- Station **B** starts sending at time t
 - If station **A** starts anytime after $t - T_{fr} \rightarrow$ collides with B's beginning
 - If station **C** starts anytime before $t + T_{fr} \rightarrow$ collides with B's end
- So B's frame is vulnerable across the whole window from $t - T_{fr}$ to $t + T_{fr}$

$$\text{Pure ALOHA vulnerable time} = 2 \times T_{fr}$$

- A collision can happen if *any other station* starts transmitting **within one full frame-time before or after** your frame starts — twice the transmission time in total.
- **Throughput** = average number of frames generated. per frame-transmission time
- Maximum throughput occurs at $G = 1/2$: even under the *best* traffic load, only **18.4%** of frames get through successfully

$$S = G \times e^{-2G}$$

$$S_{max} = \frac{1}{2e} = 0.184$$

- **Slotted ALOHA:**

- Pure ALOHA has a large vulnerable time ($2 \times T_{fr}$) because stations can send at *any* moment
- Fix: divide time into **slots of T_{fr} seconds** — stations may only transmit at the **start of a slot**
- If a station misses the slot start, it must **wait for the next slot**
- Collisions can still happen if two stations start at the same slot boundary
- Result: vulnerable time is **cut in half** → Vulnerable time = T_{fr}

- **Throughput — Slotted ALOHA:**

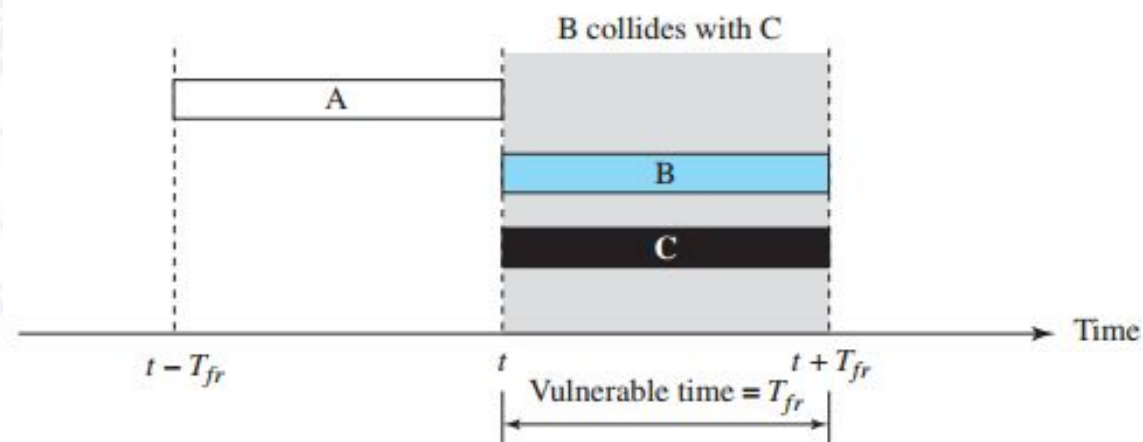
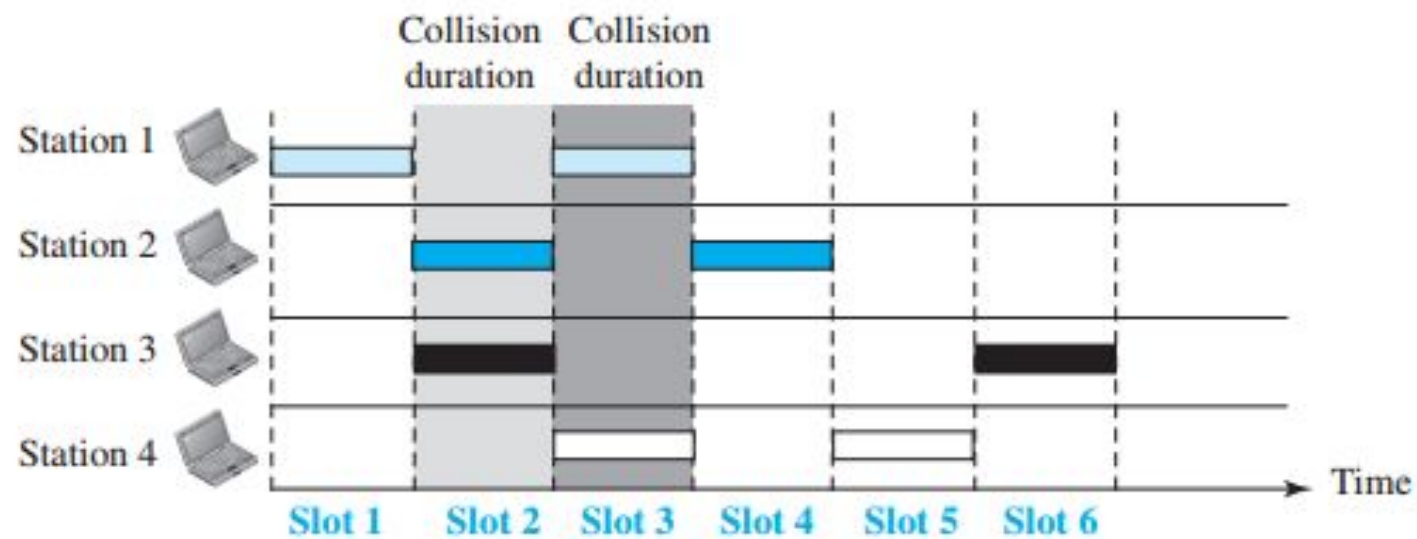
- Maximum throughput **$G = 1$** : up to **36.8%** of frames get through successfully — double the efficiency of pure ALOHA

(e = euler's no 2.71828)

$$S = G \times e^{-G}$$

$$S_{max} = \frac{1}{e} = 0.368$$

- Best performance occurs when exactly **one frame** is generated per slot (vulnerable) time, with no competing frames



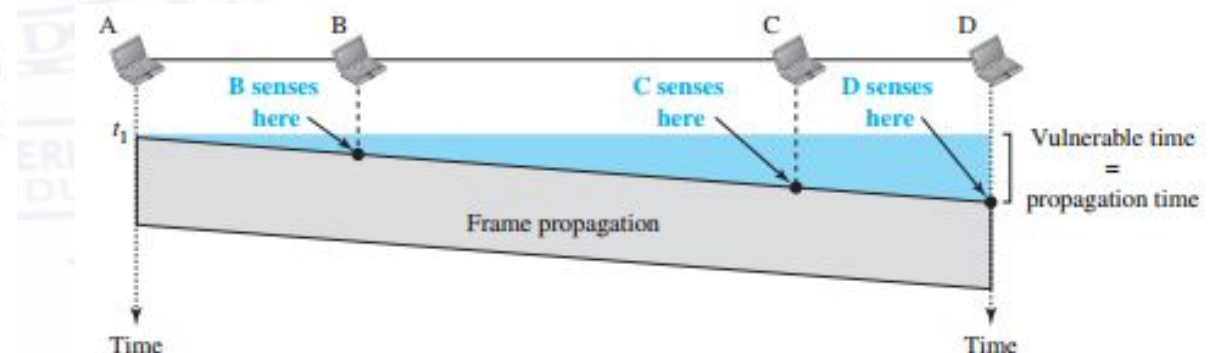
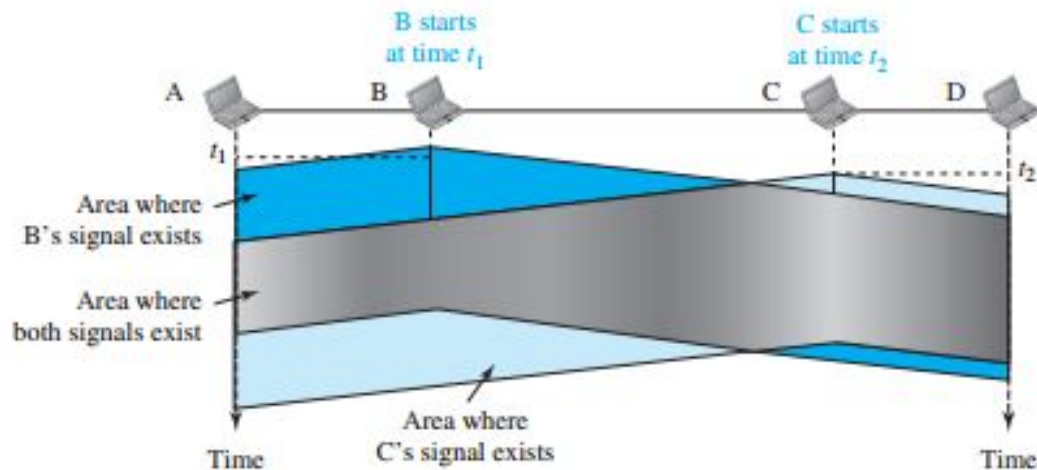
Carrier sense multiple access (CSMA)

- Goal: reduce collisions (though not eliminate them) → improve performance
- Core principle: **"sense before transmit"** / **"listen before talk"**
- Each station **listens to the medium first**; only sends if it's idle
- **Why collisions can still happen:** propagation delay
 - A station's first bit takes time to physically reach every other station
 - Another station may sense the medium *before* that bit arrives, find it idle, and transmit too → collision
- Example:
 - Station **B** senses idle at time t_1 → sends
 - Station **C** senses at time t_2 (before B's signal reaches it) → also sends
 - Both signals collide and are destroyed

- Vulnerable Time — CSMA:

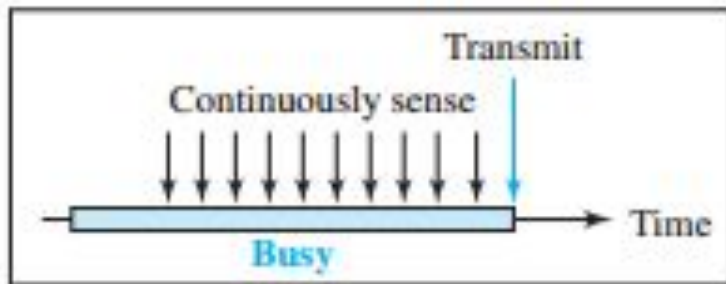
$$\text{Vulnerable time} = T_p \text{ (propagation time)}$$

- T_p = time for a signal to travel from one end of the medium to the other
- **Worst case:** the farthest station only hears the first bit after the full propagation delay — until then, it may transmit and cause a collision
- Once every station has heard the first bit, no new collision can start (they'll sense "busy" and hold off)

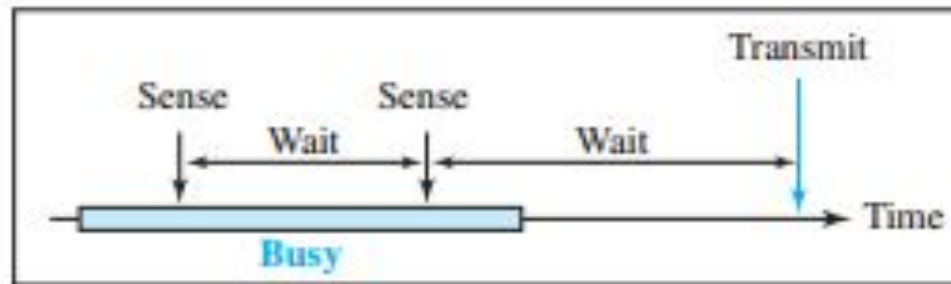


- **Persistence Methods** — What to do when the channel is busy or idle
- Three strategies answer this: **1-persistent**, **nonpersistent**, and **p-persistent**.
- **1-Persistent:**
 - Continuously sense the line; the moment it's idle → **send immediately** (probability 1)
 - Simple, but **highest collision risk** — multiple stations may all find it idle at once and transmit together
 - Used by **Ethernet**
- **Nonpersistent:**
 - If idle → send immediately
 - If busy → **wait a random time**, then sense again
 - **Reduces collisions** (stations unlikely to wait the same random time), but **reduces efficiency** — channel sits idle even when stations have data ready
- **p-Persistent:**
 - Used when the channel has **time slots $\geq$ maximum propagation time**

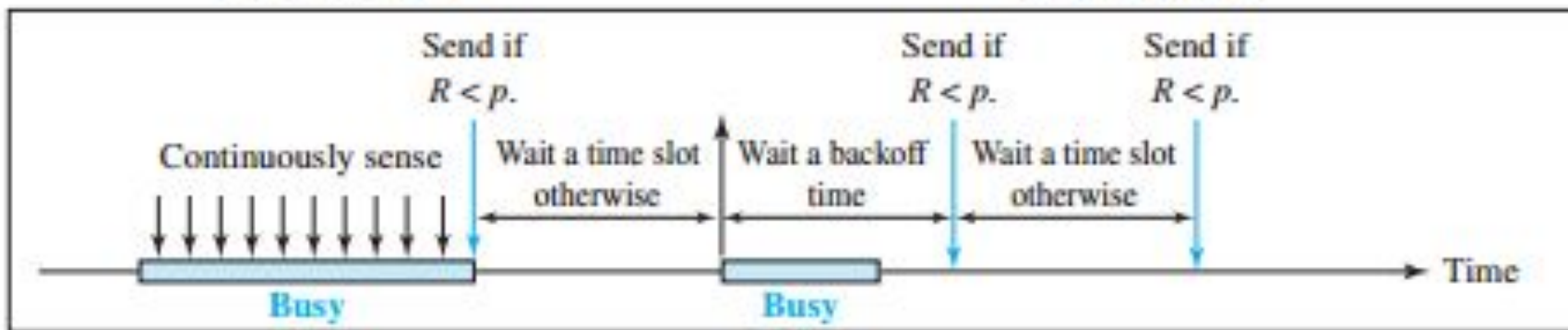
- Combines the best of both other methods:
 - If idle, transmit with **probability p**
 - With probability **$q = 1 - p$** , wait for the next slot and recheck
 - Still idle $\rightarrow$ repeat step 1
 - Busy $\rightarrow$ treat it as a collision and apply **back-off**



a. 1-Persistent



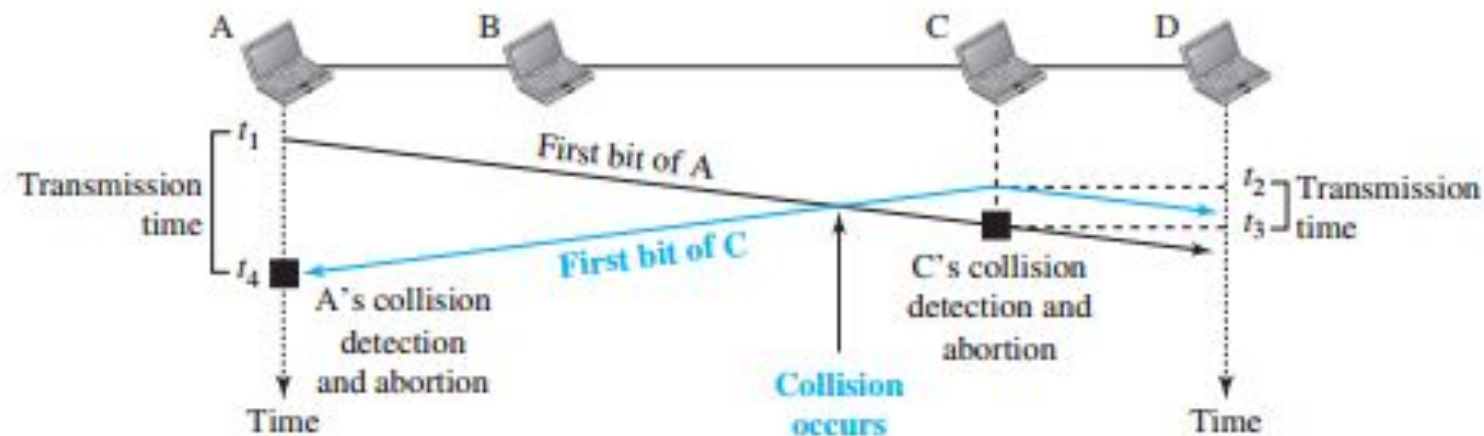
b. Nonpersistent

c. p -Persistent

Carrier sense multiple access with collision detection (CSMA/CD)

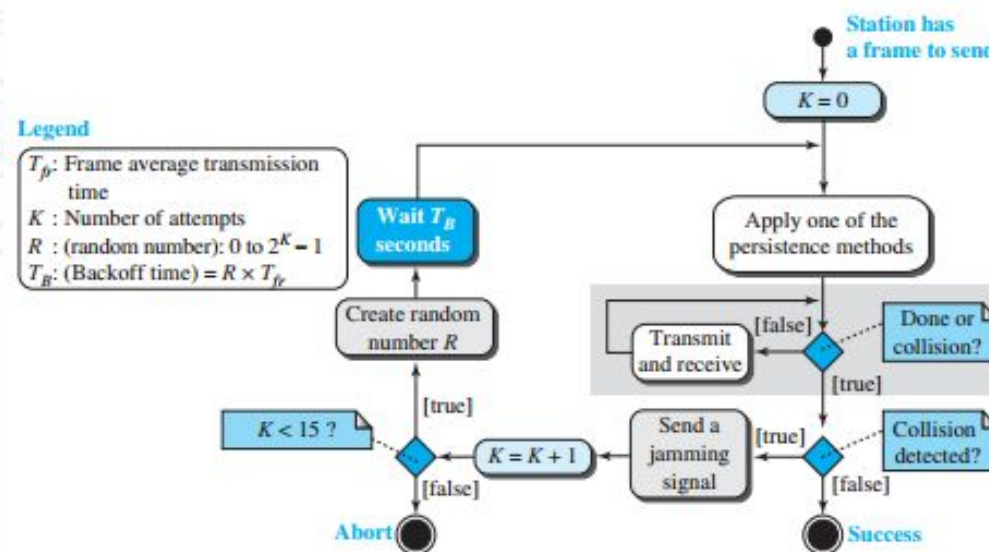
- Method: a station **monitors the medium while sending**
 - Transmission successful → done
 - Collision detected → **resend the frame**
- Example — Stations A and C collide
 - At t_1 , A starts sending; senses line idle at that moment
 - At t_2 , C hasn't yet received A's first bit → also senses idle → starts sending
 - Collision occurs somewhere between them
 - C detects the collision at t_3 (on receiving A's first bit) → **aborts immediately**
 - A detects the collision at t_4 (on receiving C's first bit) → **aborts immediately**
 - So: A transmits for duration $(t_4 - t_1)$, C transmits for duration $(t_3 - t_2)$

- Minimum Frame Size:
 - A station must be able to **detect a collision before finishing** its frame — it doesn't keep a copy or monitor the line afterward
 - **Requirement:** frame transmission time T_{fr} must be $\geq 2 \times T_p$ (twice the max propagation time)
 - **Why:** worst case — two stations at maximum distance apart. The collision signal takes T_p to reach the far station, and its effect takes another T_p to travel back. So the first station must **still be transmitting after $2 \times T_p$** to guarantee it detects the collision.

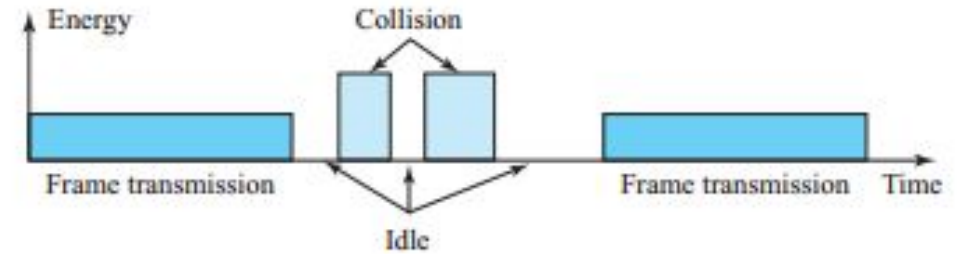


CSMA/CD Procedure — Key Differences from ALOHA:

1. **Persistence check first** — station applies a persistence method (nonpersistent, 1-persistent, or p-persistent) before sending, to sense the channel
2. **Continuous monitoring** — unlike ALOHA (send whole frame → check for ACK), CSMA/CD **transmits and listens simultaneously** in a loop, watching for either:
 1. Transmission complete → **success**
 2. Collision detected → **abort**
3. **Jamming signal** — on collision, station sends a short jamming signal so **all other stations** immediately know a collision occurred
4. **Back-off & retry** — after collision: generate random number R , wait $T_B = R \times T_p$, retry (up to **$K < 15$** attempts, else **abort**)



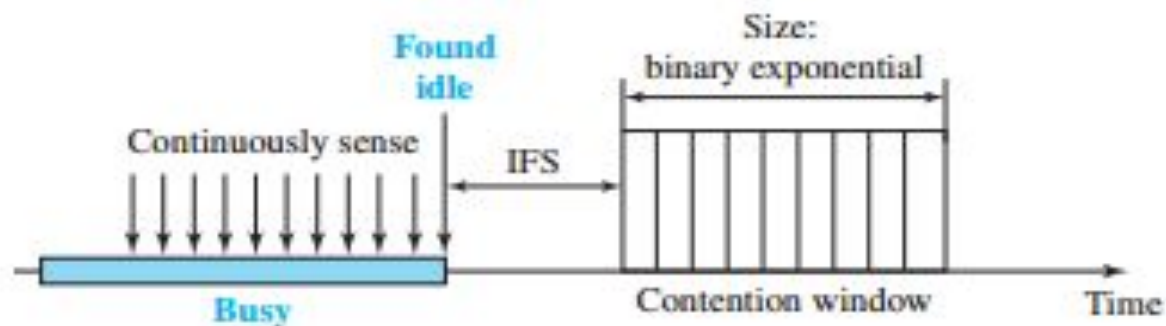
- Energy Levels on the Channel:
 - **Zero** → channel idle
 - **Normal** → one station transmitting successfully
 - **Abnormal** ($\approx$ double normal) → collision occurring
 - Stations monitor energy level to know: idle / busy / collision
- Throughput
 - Better than pure or slotted ALOHA
 - **1-persistent**: max throughput $\approx$ **50%** (at $G = 1$)
 - **Nonpersistent**: max throughput up to **90%** (G between 3–8)
- Eg: **Traditional Ethernet**
 - Classic **10 Mbps** LAN protocol using CSMA/CD with the **1-persistent** method
 - Later Ethernet versions moved away from CSMA/CD



Carrier sense multiple access with collision avoidance (CSMA/CA)

- Collisions are avoided through the use of CSMA/CA's three strategies:
 - the interframe space,
 - the contention window, and
 - Acknowledgments
- **Interframe Space (IFS)**
 - Even when the channel looks **idle**, a station **waits** before sending — this wait is called the **IFS**
 - IFS gives that distant signal **time to arrive** before this station commits to sending
 - After IFS, if the channel is *still* idle, the station still needs to wait through the **contention window**
 - IFS can also be used to **prioritize** stations/frame types — a **shorter IFS = higher priority**

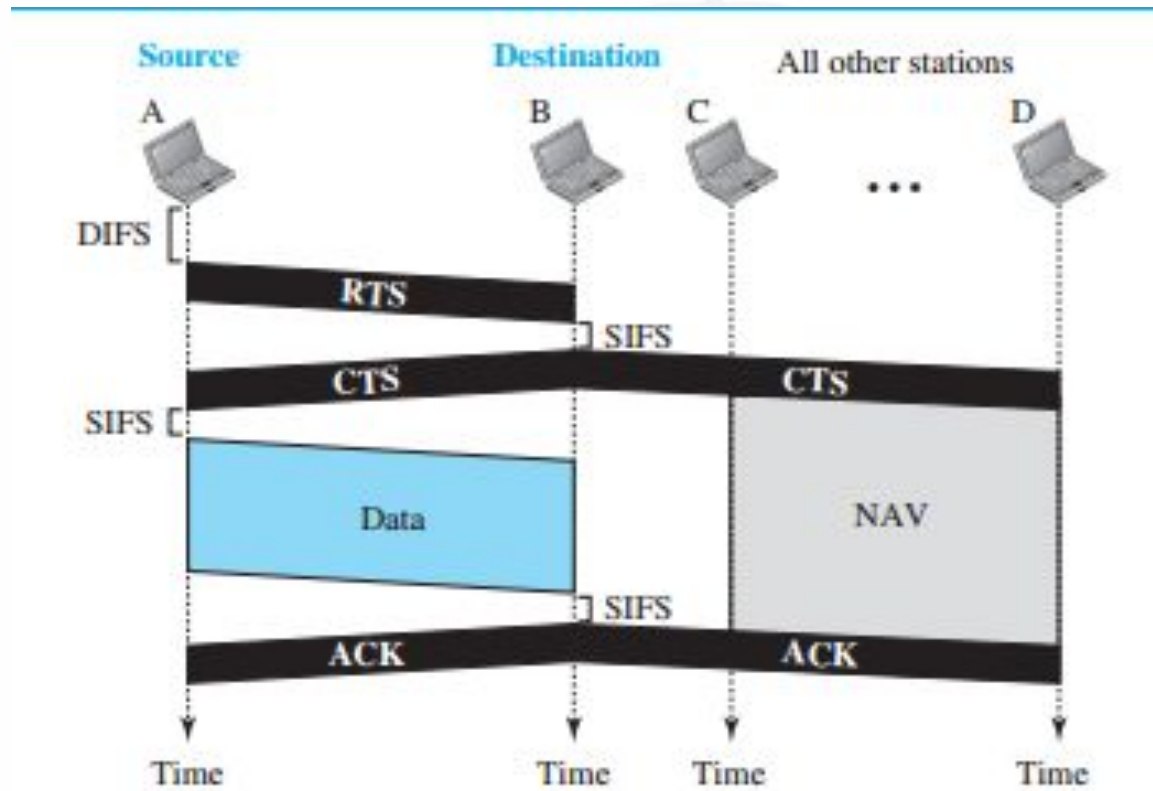
- **Contention Window:**
 - A time period divided into **slots**; a ready station picks a **random number of slots** to wait
 - Number of slots follows **binary exponential backoff**: starts at 1 slot, **doubles** each time the channel isn't found idle
 - Similar to **p-persistent** method, but the wait duration is randomized instead of a fixed probability
 - Station **senses the channel after each slot**:
 - Idle → timer continues counting down
 - Busy → timer **pauses** (doesn't restart) and resumes when idle again
 - This gives **priority to the station that has already waited longest**



- **Acknowledgment:**
 - Despite IFS + contention window, collisions can still occur, and data can still get corrupted
 - **Positive ACK + timeout timer** together guarantee the receiver has confirmed successful frame receipt
- **Frame Exchange Time Line:**
 - **Sense the medium** — source station checks the energy level at the carrier frequency
Uses a persistence strategy with **backoff** until the channel is idle
 - Once idle, station waits **DIFS** (DCF Interframe Space), then sends a control frame: **RTS** (Request to Send)
 - Destination waits **SIFS** (Short Interframe Space), then replies with **CTS** (Clear to Send) — signals it's ready to receive
 - Source waits **SIFS**, then sends the actual **Data**
 - Destination waits **SIFS**, then sends an **ACK** confirming successful receipt

Exchange sequence: Sense → DIFS → RTS → SIFS → CTS → SIFS → Data → SIFS → ACK

- **ACK is essential in CSMA/CA** — unlike CSMA/CD, a station **cannot detect collisions** while transmitting, so it has no way to confirm delivery without an explicit ACK
- **All other stations** hear the RTS/CTS exchange and set their **NAV (Network Allocation Vector)** — they stay silent for the announced duration, avoiding collisions entirely



- Network Allocation Vector (NAV):
 - The RTS frame includes the **duration** the channel will be occupied
 - Every other station that hears this sets a timer — the **NAV** — counting down that duration
 - Before sensing the physical medium, a station **first checks its NAV** — if not yet expired, it assumes the channel is busy without needing to sense it directly
- Collision During Handshaking:
 - Collisions can still happen when **two+ stations send RTS at the same time** (during the RTS/CTS handshake itself)
 - Since there's **no collision-detection mechanism**, the sender infers a collision simply by **not receiving a CTS** back
 - Response: apply the **backoff strategy** and retry

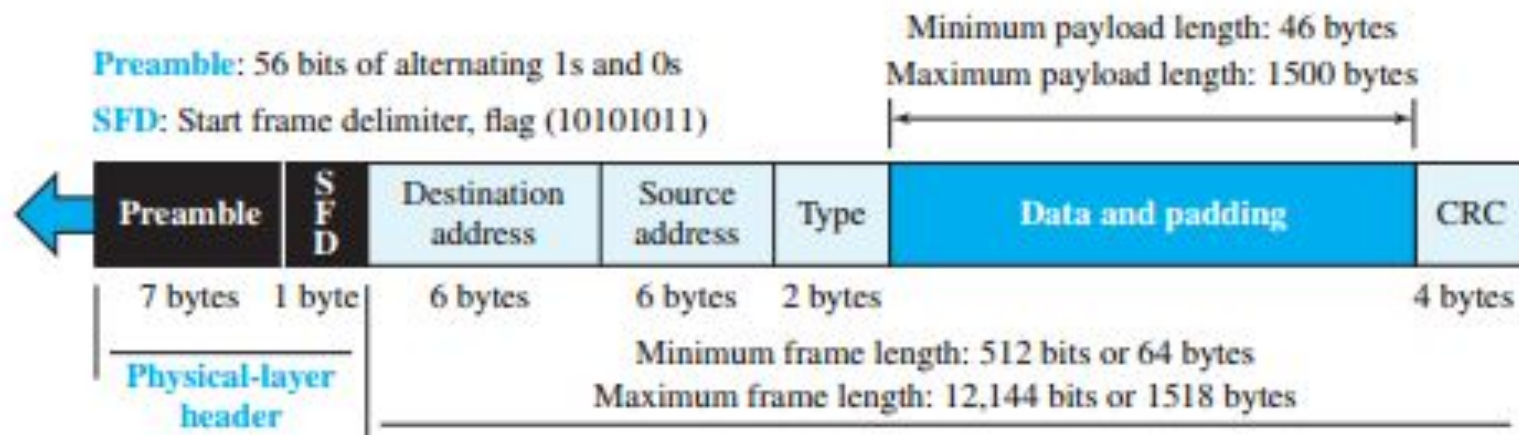
- Hidden-Station Problem — Solved by RTS/CTS:
 - Example: B's RTS reaches A, but **not C** (C is "hidden" from B)
 - However, **both B and C are within range of A**
 - A's **CTS** (which includes the data-transmission duration) reaches **C** too
 - C now knows the channel is occupied and **refrains from transmitting** until that duration ends
- CSMA/CA and Wireless Networks
 - CSMA/CA was designed mainly for **wireless** use
 - The basic version above isn't sophisticated enough for all wireless issues (e.g., hidden/exposed terminals)
 - These are addressed with additional **handshaking features**

Standard Ethernet

- A standard method for **connecting devices and transferring data over a wired local area network (LAN)**
- It is **Connectionless and Unreliable** Service:
 - **Connectionless:** each frame sent is independent — no relation to the frame before or after; no connection creation/termination
 - **Unreliable:** if a frame is corrupted during transmission (detected via **CRC-32**), the receiver **silently discards it** — no notification sent
 - Recovery is left to **higher-layer protocols**, not Ethernet:
 - If the transport layer uses **UDP** (also connectionless) → frame loss simply goes unnoticed
 - If the transport layer uses **TCP** → sender doesn't get an ACK for that segment, so it **automatically retransmits**

Frame format:

- **Preamble** — 7 bytes (56 bits) of alternating 0s and 1s; lets the receiver **sync its clock**. Added at the physical layer, not formally part of the frame.
- **Start Frame Delimiter (SFD)** — 1 byte (10101011); signals the frame is about to begin. The last 2 bits (11) alert the receiver that the **next field is the destination address**. Also added at the physical layer.
- **Destination Address (DA)** — 6 bytes (48 bits); the link-layer address of the receiving station(s) — could be unicast, multicast, or broadcast.
- **Source Address (SA)** — 6 bytes; link-layer address of the sender.



- **Type** — identifies the **upper-layer(N/W) protocol** carried inside (IP, ARP, OSPF, etc.) — used for multiplexing/demultiplexing.
- **Data** — the actual payload, **46 to 1500 bytes**
 - If data > 1500 bytes → must be **fragmented** into multiple frames
 - If data < 46 bytes → **padded** with extra 0s (padding must be added/removed manually since the frame doesn't track original length)
- **CRC** — error-detection field (**CRC-32**), calculated over addresses, type, and data. If the receiver's recalculated CRC isn't zero → frame is **corrupted and discarded**.

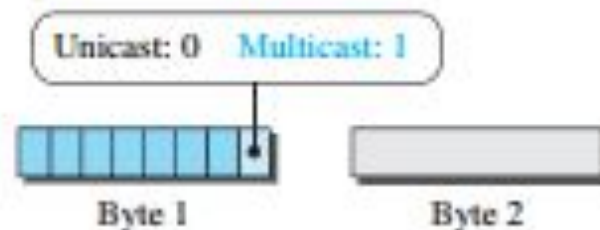
Frame length:

- Minimum frame = **64 bytes (512 bits)**
 - Header + trailer = 18 bytes (6 SA + 6 DA + 2 type + 4 CRC)
 - Minimum data length = $64 - 18 = 46$ bytes → padding added if data is smaller
- Maximum frame (excluding preamble/SFD) = **1518 bytes**
 - Maximum data (payload) = $1518 - 18 = 1500$ bytes

	Minimum	Maximum
Frame length	64 bytes	1518 bytes
Data length	46 bytes	1500 bytes

Addressing:

- Every station has a **Network Interface Card (NIC)**, which provides its **link-layer (MAC) address**
- MAC address = **6 bytes (48 bits)**, written in **hexadecimal**, separated by colons
Example: 4A:30:10:21:10:1A
- **Transmission of Address Bits:** Addresses are sent **byte by byte, left to right**
 - But **within each byte**, bits go **right to left**
- **Unicast, Multicast, and Broadcast Addresses:**
 - **Source address** is always **unicast** (comes from one station)
 - **Destination address** can be:
 - **Unicast** — least significant bit of the first byte = **0**
 - **Multicast** — that bit = **1**
 - **Broadcast** — special case of multicast; all **48 bits = 1** (goes to every station on the LAN)

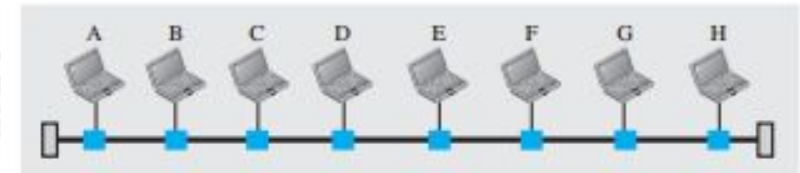


Define the type of the following destination addresses

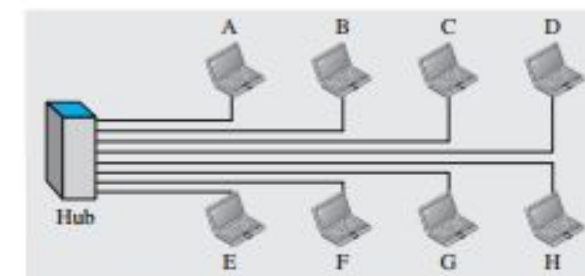
- 4A:30:10:21:10:1A
- 47:20:1B:2E:08:EE
- FF:FF:FF:FF:FF:FF

Distinguishing Unicast, Multicast, and Broadcast:

- Standard Ethernet uses **coaxial cable (bus topology)** or **twisted-pair cables with a hub (star topology)**
- Physically, transmission is **always broadcast** — regardless of whether the intent is unicast, multicast, or broadcast
 - Bus topology:** all stations receive whatever A sends to B
 - Star topology:** the hub is passive — it doesn't check destination addresses, just **regenerates and forwards** the frame to all stations except the sender, effectively **flooding** the network
 - Unicast:** all stations receive it, but only the **intended recipient keeps it**; everyone else discards it
 - Multicast:** all stations receive it; only stations that are **members of that group** keep and handle it; the rest discard it



a. A LAN with a bus topology using a coaxial cable



b. A LAN with a star topology using a hub

Legend

- A host (of any type)
- A hub
- A cable tap
- A cable end
- Coaxial cable
- Twisted pair cable

$$T_p = \frac{\text{Distance}}{\text{Propagation speed}}$$

$$T_t = \frac{\text{Frame size}}{\text{Transmission rate}}$$

Efficiency of Standard Ethernet:

- **Efficiency** = ratio of time a station spends actually sending data to the total time the medium is occupied

$$\text{Efficiency} = \frac{1}{1 + 6.4a}$$

- **a** = number of frames that can fit on the medium at once = propagation delay ÷ transmission delay
 - **Propagation delay** = time for a signal to reach the end of the medium
 - **Transmission delay** = time to send the whole frame
- Eg: Given: transmission rate = 10 Mbps, cable length = 2500 m, frame size = 512 bits, signal speed = 2×10^8 m/s

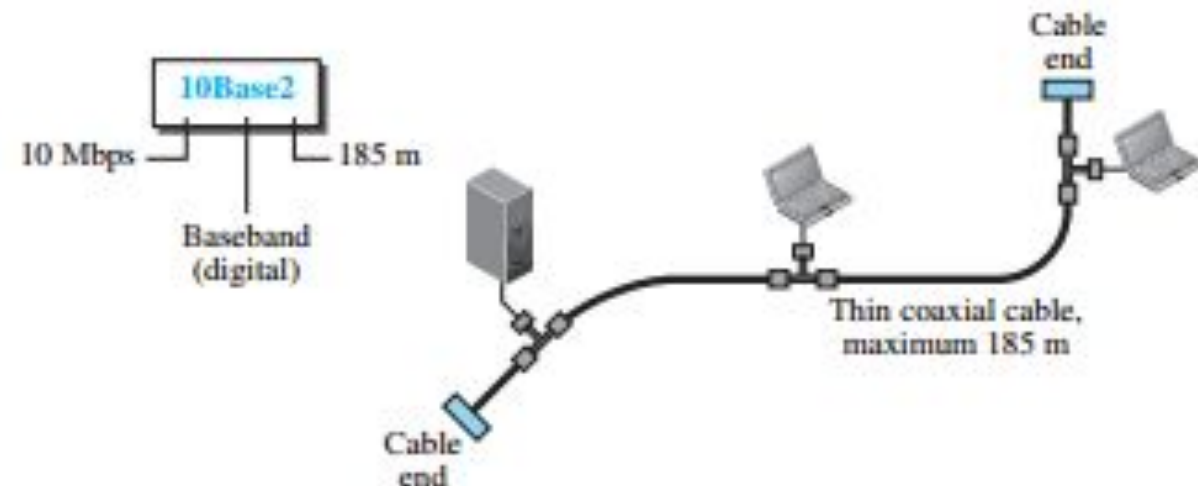
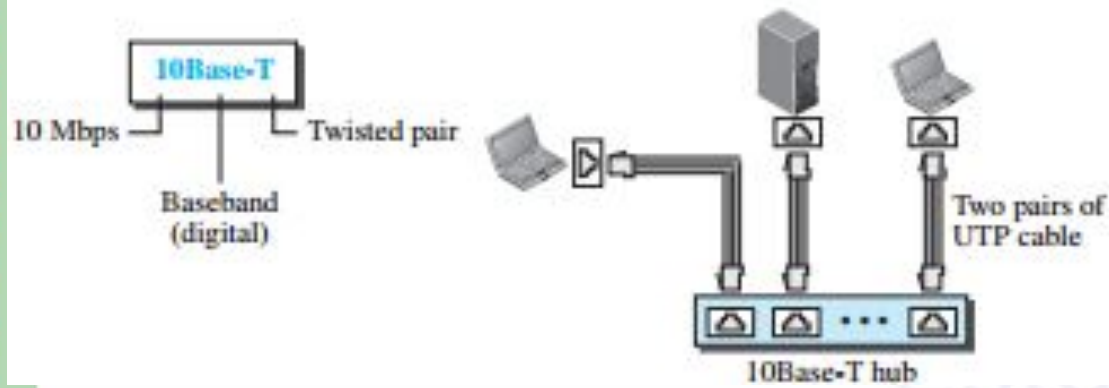
Quantity	Calculation	Result
Propagation delay	$2500 / (2 \times 10^8)$	12.5 μ s
Transmission delay	$512 / 10^7$	51.2 μ s
a	$12.5 / 51.2$	0.24
Efficiency	$1 / (1 + 6.4 \times 0.24)$	$\approx 39\%$

Implementation:

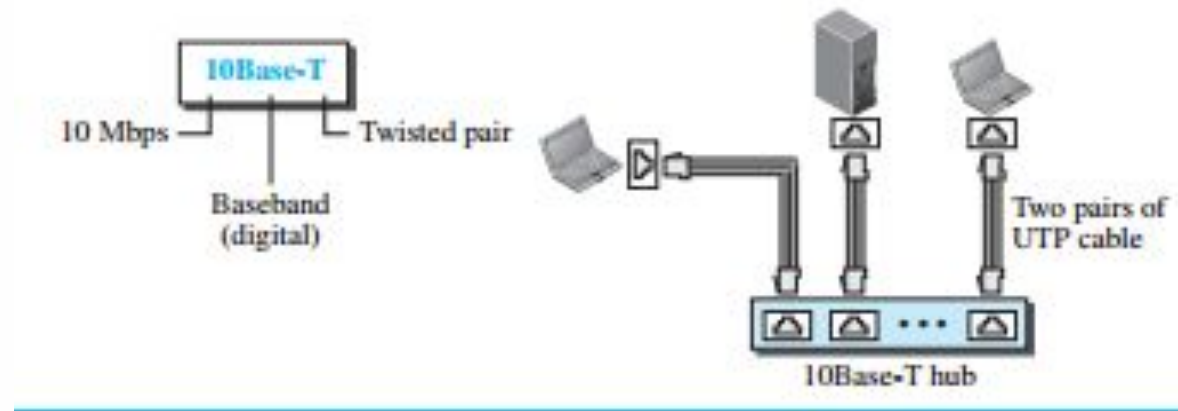
- **Encoding and Decoding :**
 - All implementations use **digital signaling at 10 Mbps**
 - Data is converted to a digital signal using **Manchester encoding**
 - Manchester encoding is **self-synchronizing** — provides a signal transition at every bit interval, so sender and receiver stay in sync without a separate clock signal
- **10Base5: Thick Ethernet ("Thicknet") :**
 - First Ethernet implementation; nicknamed for its thick, stiff cable — roughly the size of a garden hose
 - Uses a **bus topology** with **thick coaxial cable**
 - Connects via an external **transceiver**, linked to the station via a separate **transceiver cable** (max 50 m) — provides separate send/receive paths, so **collisions can only occur in the coaxial cable**
 - **Max cable length: 500 m** (beyond that, signal degrades too much)
 - Longer networks can chain **up to 5 segments** (500 m each) using **repeaters**

Implementation	Medium	Medium Length	Encoding
10Base5	Thick coax	500 m	Manchester
10Base2	Thin coax	185 m	Manchester
10Base-T	2 UTP	100 m	Manchester
10Base-F	2 Fiber	2000 m	Manchester

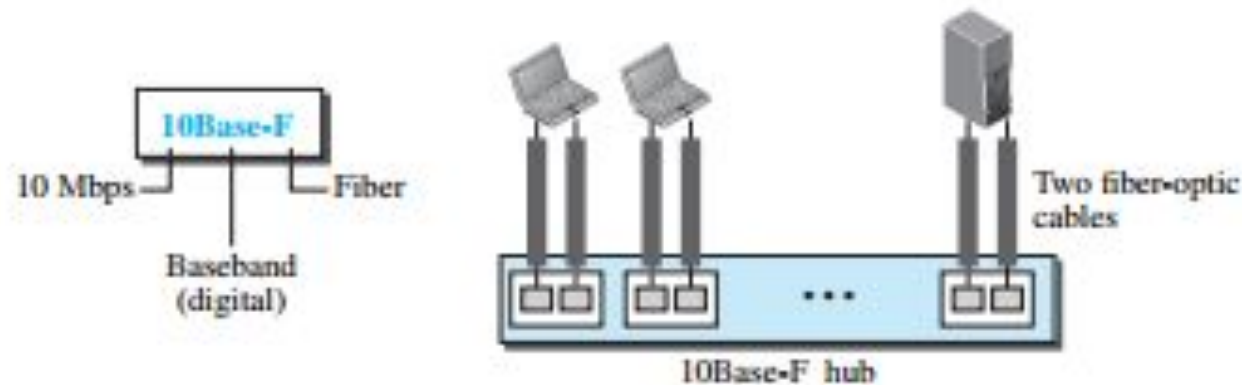
- **10Base2: Thin Ethernet ("CheaperNet"):**
 - Also uses **bus topology**, but with **thinner, more flexible cable**
 - The **transceiver is built into the NIC** inside the station (no external transceiver needed)
 - **More cost-effective** than 10Base5 — cheaper cable, simpler tee connections, easier installation
 - **Max cable length: 185 m** (~200 m) due to higher signal attenuation in thin coax
 - Collisions occur within the thin coaxial cable itself



- 10Base-T: Twisted-Pair Ethernet:
 - Uses a **physical star topology** — stations connect to a **hub** via **two pairs of twisted cable** (UTP)
 - The two pairs create **separate send and receive paths** between station and hub
 - **Collisions happen inside the hub** — the hub essentially takes over the role coax played in 10Base5/10Base2
 - **Max cable length: 100 m**, to minimize signal attenuation over twisted-pair cable

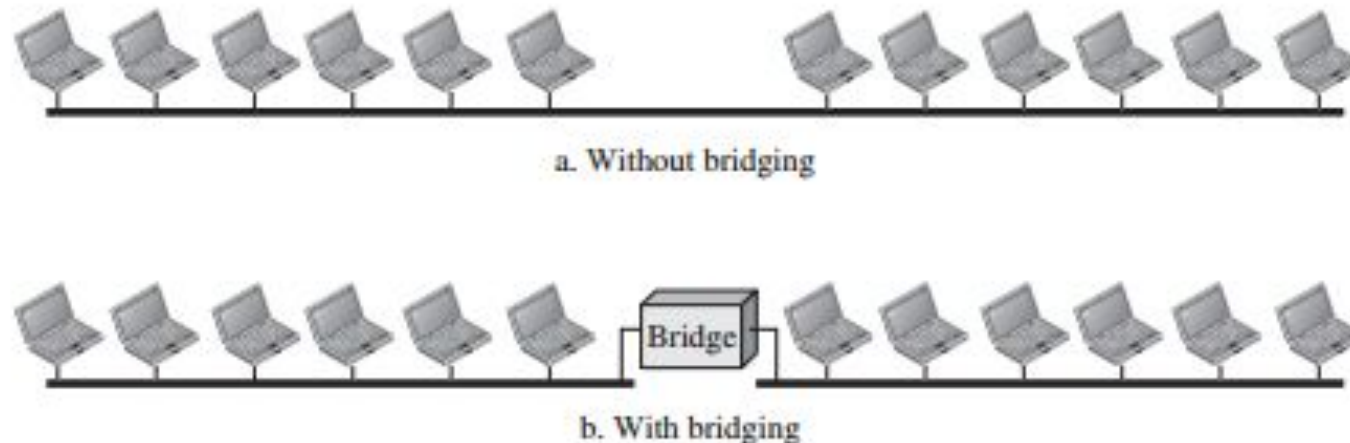


- 10Base-F: Fiber Ethernet:
 - Also uses **star topology**, connecting stations to a hub — but over **fiber-optic cable** instead of copper
 - Uses **two fiber-optic cables** per station (send and receive paths)
 - Most common variant: **10Base-F**
 - Offers much greater cable length than the copper-based implementations, since fiber suffers far less attenuation

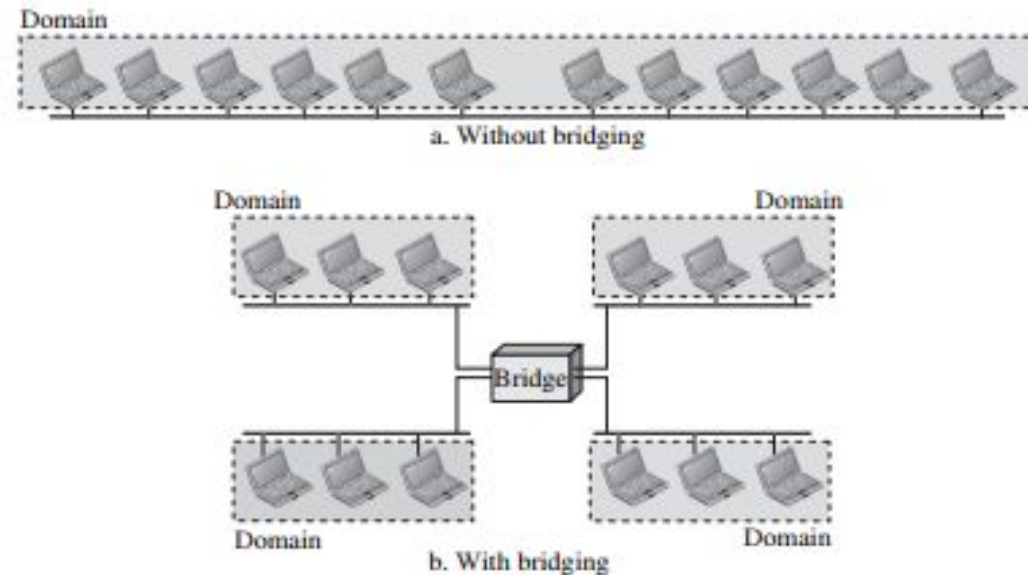


Changes in the Standard:

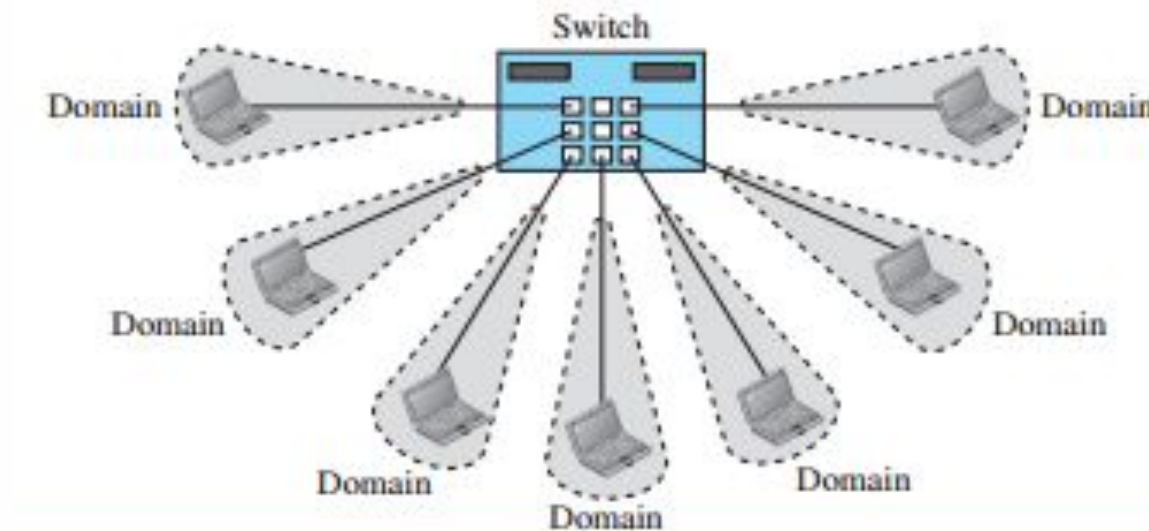
- Bridged Ethernet:
 - First evolution step: dividing a LAN using **bridges**
 - Two effects: **raises bandwidth** and **separates collision domains**
 - Without a bridge, all stations share the full 10 Mbps capacity — if 2 stations are active, each effectively gets ~5 Mbps
 - A bridge splits the network into **independent segments**, each with its own 10 Mbps capacity, shared among fewer stations



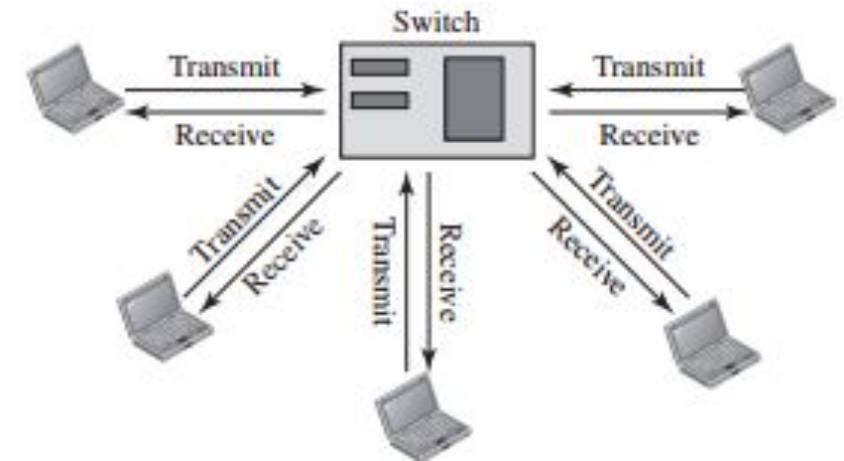
- Separating Collision Domains:
 - A bridge shrinks each **collision domain**, reducing collision probability
 - Example: 12 stations without bridging = 1 large domain; with a bridge = smaller domains contending independently



- Switched Ethernet:
 - Extends bridging further: instead of a few bridge ports, use an **N-port switch** — one port per station
 - Bandwidth is now shared only between **each station and the switch** (not with other stations)
 - Collision domain shrinks to just N domains (essentially one per station)
 - A **layer-2 switch** = a multi-port bridge with more sophistication



- Full-Duplex Ethernet:
 - 10Base5/10Base2 were **half-duplex** — a station can send OR receive, not both at once
 - **Full-duplex switched Ethernet** doubles capacity (10→20 Mbps) using **two separate links** per station: one to transmit, one to receive
 - No Need for CSMA/CD:
 - With dedicated point-to-point links and no shared medium, there's **no possibility of collision**
 - Carrier sensing and collision detection become unnecessary and can be **turned off**
 - MAC Control Layer:
 - Standard Ethernet has **no flow/error control** at the MAC sublayer — purely connectionless
 - Full-duplex switched Ethernet adds a new sublayer, **MAC control**, between LLC and MAC, to provide flow and error control



What is the hexadecimal equivalent of the following Ethernet address?

01011010 00010001 01010101 00011000 10101010 00001111

How does the Ethernet address 1A:2B:3C:4D:5E:6F appear on the line in binary?

If an Ethernet destination address is 07:01:02:03:04:05, what is the type of the address (unicast, multicast, or broadcast)?

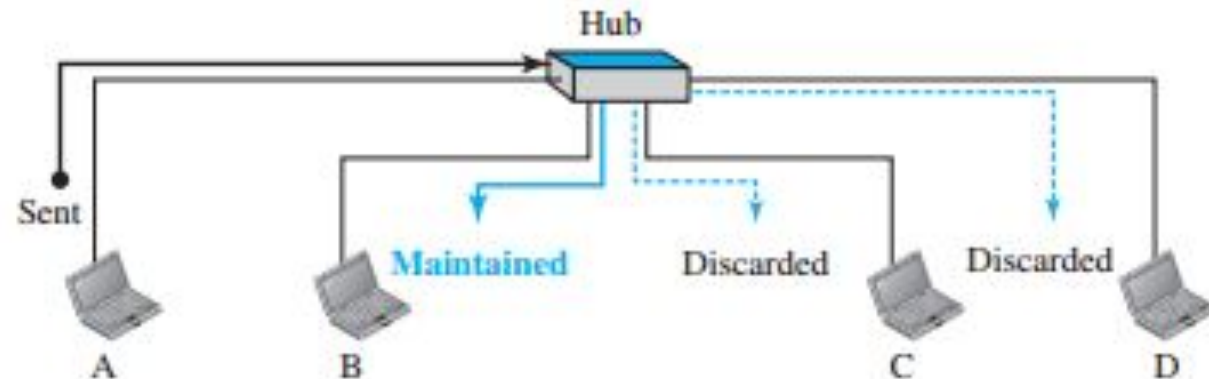
An Ethernet MAC sublayer receives 42 bytes of data from the upper layer. How many bytes of padding must be added to the data?



Connecting devices

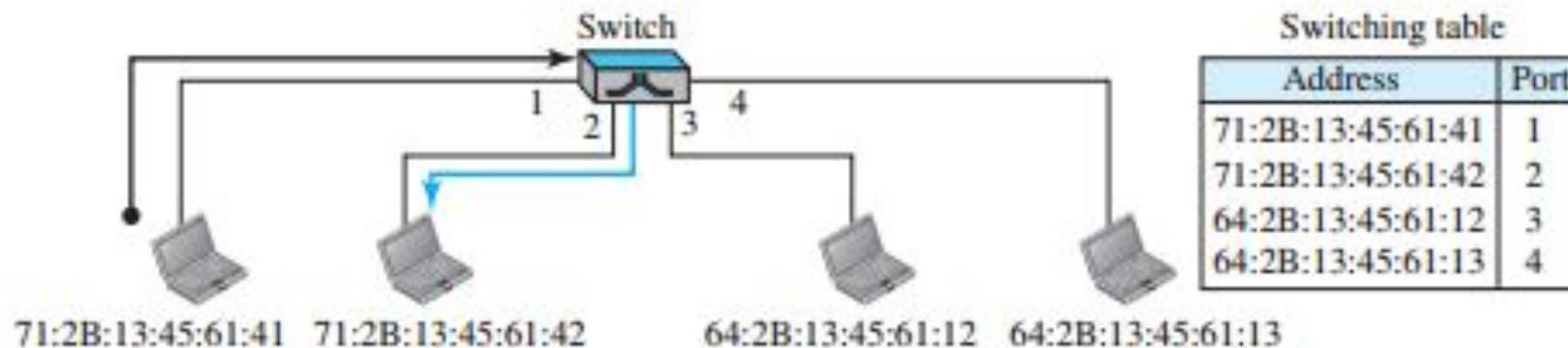
▪ Hub:

- A **hub** operates only at the **physical layer**
- A **repeater** receives a signal and **regenerates/retimes** it before attenuation corrupts the data — used historically to extend bus-topology LAN segments beyond cable length limits
- In modern **star topology**, a hub acts as both the connecting point *and* a repeater — a **multiport repeater**
- **Eg:** Hub **forwards the frame out of every port** except the one it arrived on (i.e., it **broadcasts**)
 - All stations receive the frame, but only the **intended recipient (B) keeps it** — the rest discard it



▪ Switches:

- Unlike a hub, a **link-layer switch** has **filtering** capability
- It checks the frame's **destination address** and decides **which specific port** to forward it through
- This decision is based on a **switching table** (address → port mapping)
- **A link-layer switch does not change the MAC addresses in a frame** — it only decides where to forward it.
- **Example:** In a 4-station LAN, a frame for 71:2B:13:45:61:42 arriving at port 1 is forwarded **only out port 2** — no need to send it through the other ports.



- **Transparent Switches:**
 - A **transparent switch** is invisible to the stations — no reconfiguration needed when switches are added/removed
 - Per **IEEE 802.1d**, a transparent switch must:
 - **Forward frames** from one station to another
 - **Build its forwarding table automatically** by learning from frame movements
 - **Prevent loops** in the network
- **Advantages of Switches (over Hubs):**
 - Collision Elimination
 - No shared medium → **no collisions**
 - No need for carrier sensing or collision detection
 - Each host can transmit **at any time** → higher effective bandwidth per host
 - Connecting Heterogeneous Devices
 - Can bridge devices using **different physical-layer protocols** (data rates, media types)
 - E.g., one side runs **10 Mbps over twisted-pair**, the other runs **100 Mbps over fiber** — as long as the **data-link layer format stays the same**, the switch can connect them

- Routers:
 - **A router is a three-layer device** — physical, data-link, and network layers.
 - As a **physical-layer** device: regenerates the signal
 - As a **link-layer** device: checks physical (MAC) addresses
 - As a **network-layer** device: checks **network-layer (IP) addresses**
 - **Routers connect networks** — linking independent networks together forms an **internetwork (internet)**.
 - Three Key Differences from a Repeater/Switch:
 - A router has **both a physical (MAC) and logical (IP) address** for each interface
 - A router only acts on packets whose **link-layer destination matches the receiving interface's address**
 - A router **changes the link-layer address** (both source and destination) when forwarding a packet

- Example — Connecting Two LANs:
 - Two buildings, each with a **Gigabit Ethernet LAN** (via switches)
 - Connected using **10-Gigabit Ethernet** to a shared router
 - The **router** then connects the whole combined system to the **Internet**

